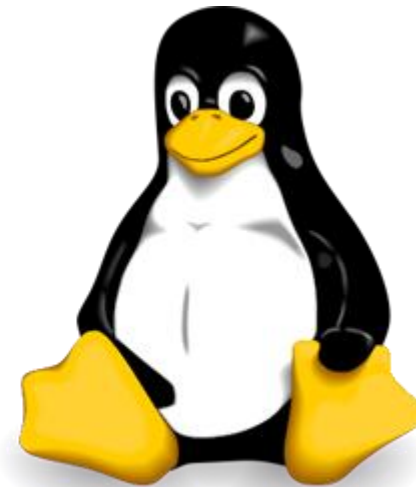


Sistemas Operacionais - Lic. Comp. 6º N

Aula 4



Aula	Tema Central (Baseado na Ementa)	Status
01	Introdução e Conceitos de Processos	Feita
02	Escalonamento de Processos	Feita
03	Sincronização de Processos	Feita
04	Alocação de Recursos e Deadlocks	Em curso
05	Gerenciamento de Memória	Planejada
06	Memória Virtual	Planejada
07	Gerenciamento de Arquivos	Planejada
08	Técnicas de E/S e Métodos de Acesso	Planejada
09	Sistemas Cliente-Servidor e Multiprocessamento	Planejada
10	Análise de Desempenho e Revisão Final	Planejada

Revisão - Aula 3

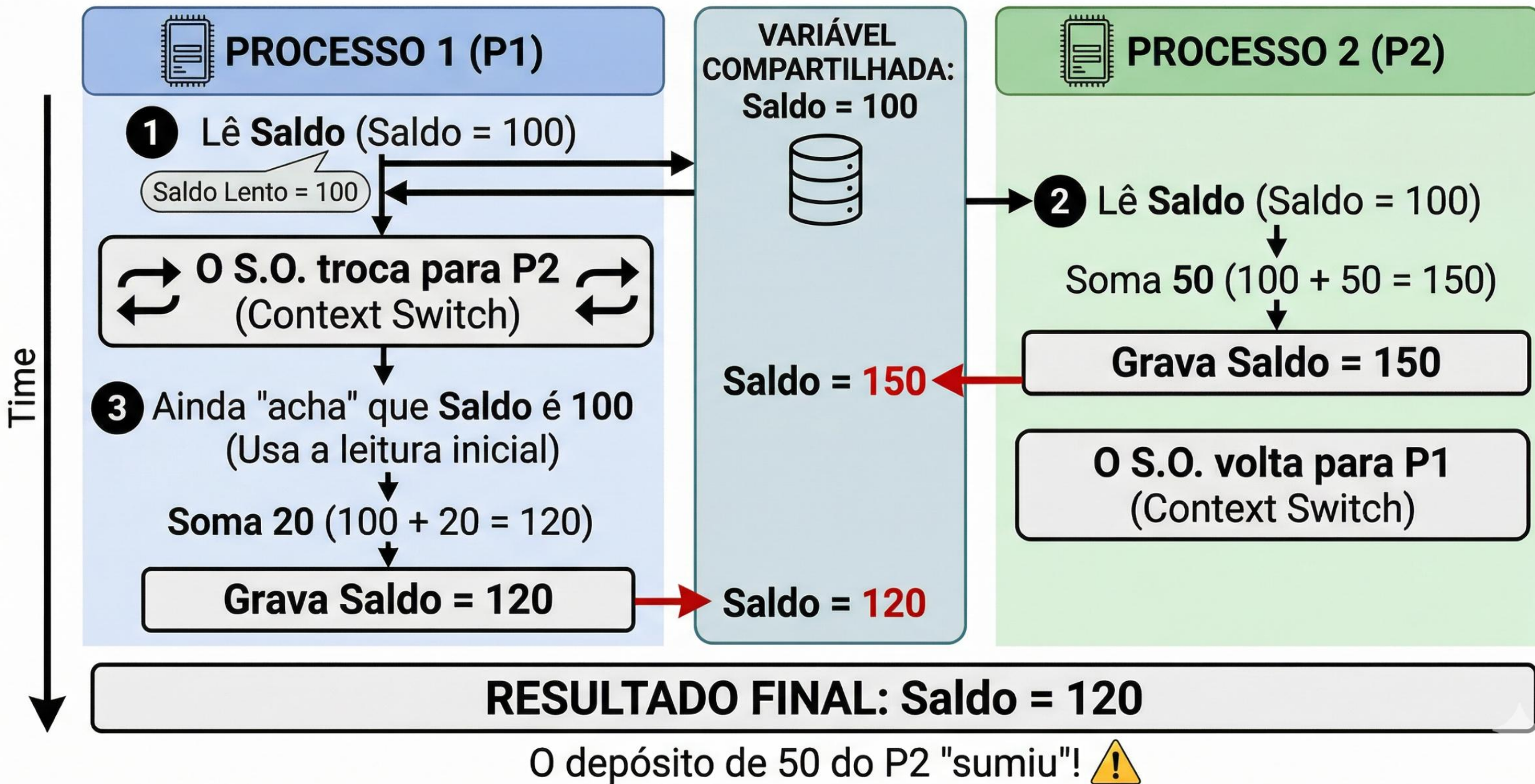
Sincronização de Processos

Condição de Corrida (Race Condition)

Definição Técnica - Condição de Corrida (Race Condition)

- Segundo Silberschatz, Galvin e Gagne: Uma condição de corrida ocorre quando vários processos acessam e manipulam os mesmos dados simultaneamente, e o resultado da execução depende da ordem específica em que o acesso ocorre.
- Segundo Tanenbaum: É a situação em que dois ou mais processos estão lendo ou escrevendo em algum dado compartilhado e o resultado final depende de quem roda precisamente quando.

DIAGRAMA DE FLUXO: CONDIÇÃO DE CORRIDA E PERDA DE ATUALIZAÇÃO (SISTEMAS OPERACIONAIS)



O Problema do Produtor-Consumidor

Ilustrar como os **semáforos** e a **exclusão mútua** **trabalham juntos** para coordenar o fluxo de dados entre processos que trabalham em velocidades diferentes.

Definição Técnica - Problema do Produtor-Consumidor

Segundo Silberschatz, Galvin e Gagne: É o problema de sincronização entre dois processos que compartilham um buffer de tamanho fixo. O produtor coloca itens no buffer e o consumidor os retira.

Segundo Tanenbaum: O desafio é garantir que o produtor não tente adicionar itens a um buffer cheio e o consumidor não tente retirar itens de um buffer vazio.

Definição Técnica - Problema do Produtor-Consumidor

Segundo Machado e Maia: Requer três semáforos: um para exclusão mútua (Mutex), um para contar as posições vazias e outro para contar as posições ocupadas no buffer.

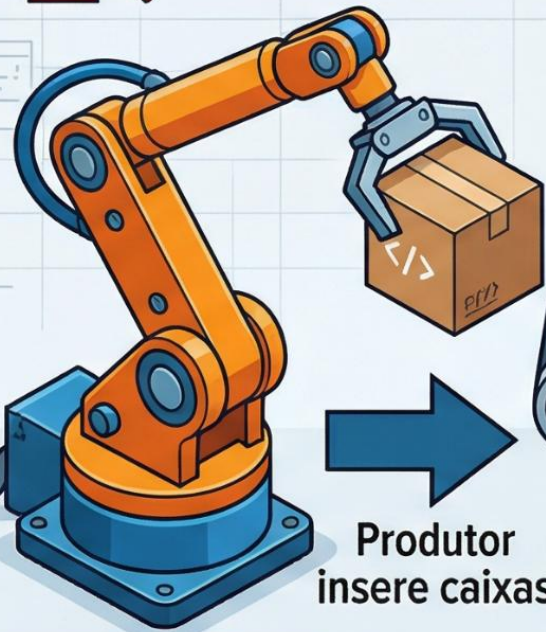
SISTEMAS OPERACIONAIS: MODELO PRODUTOR-CONSUMIDOR (LINHA DE MONTAGEM INDUSTRIAL SIMPLIFICADA)



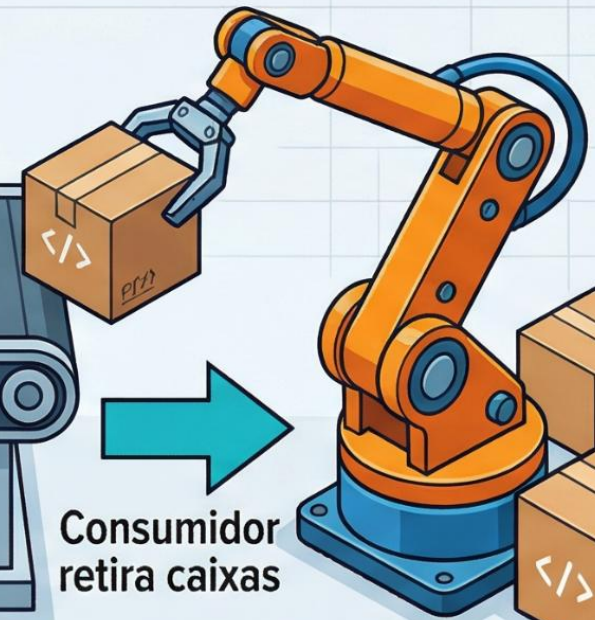
Produtor Bloqueado se
Buffer estiver Cheio (5/5)



Consumidor Bloqueado
se Buffer estiver Vazio (0/5)



Produtor
insere caixas



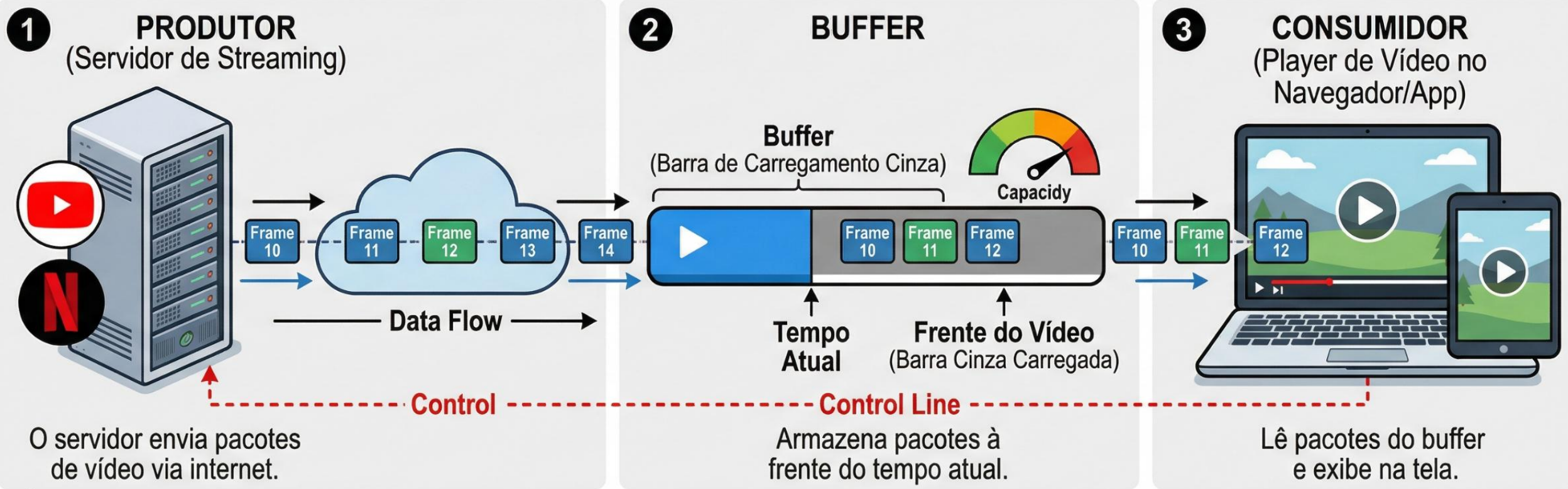
Consumidor
retira caixas

**ROBÔ
PRODUTOR**

**ROBÔ
CONSUMIDOR**

O Produtor insere dados no Buffer. O Consumidor remove dados.
O Buffer gerencia o fluxo para evitar perdas ou leituras inválidas.
Indicadores de 'Full' e 'Empty' guiam a sincronização dos processos.

SISTEMAS OPERACIONAIS: EXEMPLO PRÁTICO DE BUFFERING (PRODUTOR-CONSUMIDOR) EM STREAMING DE VÍDEO (YouTube/Netflix)



CENÁRIOS DE SINCRONIZAÇÃO



A VÍDEO TRAVA (Estado de Espera)
O player tenta ler um trecho que ainda não chegou no buffer.



B CONTROLE DE FLUXO (Buffer Cheio)
O navegador avisa o servidor para pausar o envio por um instante (não desperdiçar memória).

Aula 4

Deadlock

Deadlock nada mais é do que um erro de logística do Sistema Operacional.

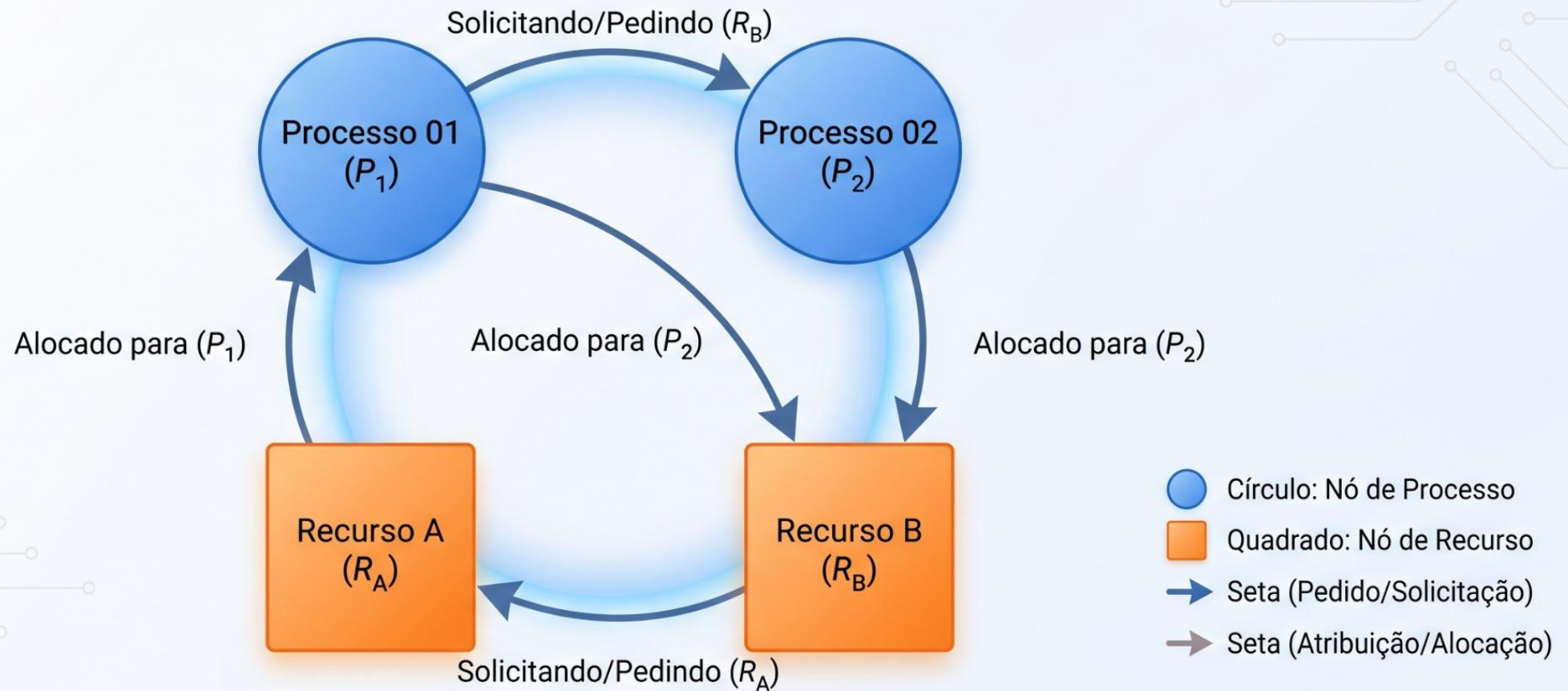
O que é um Deadlock? (Impasse)

Definição Técnica

- Segundo Silberschatz, Galvin e Gagne: Um deadlock ocorre quando cada processo em um conjunto está esperando por um evento que só pode ser causado por outro processo no mesmo conjunto. É um estado de espera mútua e eterna.
- Segundo Machado e Maia: Popularmente conhecido como "Abraço Mortal" (*Deadly Embrace*), o impasse acontece quando dois ou mais processos ficam impedidos de concluir suas tarefas porque estão bloqueados por recursos retidos um pelo outro.

GRAFO DE ALOCAÇÃO DE RECURSOS (RAG): ILUSTRAÇÃO DE DEADLOCK

Exemplo Simples de Impasse



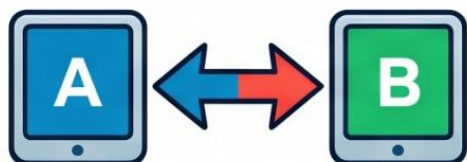
Um ciclo fechado no grafo sem recursos adicionais indica um deadlock. Neste caso, P_1 está esperando por R_B mantido por P_2 , que está esperando por R_A mantido por P_1 .



Analogia de Sistemas Operacionais: O Cruzamento Travado (DEADLOCK)

Quatro carros chegam simultaneamente no cruzamento em 'X'. Carro A bloqueado por B, B por C, C por D, D por A. Ninguém pode avançar (**bloqueio**). Ninguém pode recuar (carros atrás). O trânsito para (**IMPASSE**).

DEADLOCK: EXEMPLO PRÁTICO (CENÁRIO DE LAB)



O CONFLITO: Ambos ficam travados em uma tela preta ou “ampulheta”. O usuário acha que o PC travou, mas a verdade, os dois processos estão educadamente esperando um pelo outro para sempre.



VISTA DO USUÁRIO:
“PC Travou?”



As 4 Condições de Coffman

Para que um Deadlock aconteça, **quatro condições** específicas precisam estar presentes **ao mesmo tempo**.

Definição Técnica

1. **Exclusão Mútua:** Pelo menos um recurso deve ser mantido em modo não compartilhável (apenas um processo por vez).
2. **Posse e Espera:** Um processo deve estar segurando pelo menos um recurso e esperando para adquirir recursos adicionais que estão sendo segurados por outros processos.

Definição Técnica

3. Não Preempção (interrupção quando acaba o quantum): Os recursos não podem ser liberados à força; um processo só libera seu recurso voluntariamente após concluir sua tarefa.

4. Espera Circular: Deve existir um conjunto $\{P_0, P_1, \dots, P_n\}$ de processos esperando, onde P_0 espera por um recurso de P_1 , P_1 por P_2 , e P_n espera por um recurso de P_0 .

PREVENÇÃO DE DEADLOCK: QUEBRANDO AS QUATRO CONDIÇÕES

O SISTEMA
VOLTA A FLUIR!



EXCLUSÃO MÚTUA

Apenas um processo
usa um recurso por
vez.



POSSE E ESPERA

Um processo segura
um recurso e espera
por outros.



SEM PREEMPÇÃO

Recursos não podem
ser retirados à força.



ESPERA CIRCULAR

A Espera Circular é eliminada!
Uma força externa
quebra a cadeia de espera.

Se eliminarmos qualquer uma das quatro exigências (condições necessárias), o sistema **volta a fluir, prevenindo** o deadlock.

O JANTAR DOS FILÓSOFOS: ANALOGIA DE DEADLOCK



CONDIÇÕES DE DEADLOCK:

1. **EXCLUSÃO MÚTUA** (Cada Garfo é 1)
2. **POSSE E ESPERA** (Cada Filosofo 5
(Cada Filosofo Segura um Garfo e Espera outro)
cada Filosofo Segura um Garfo e Espera outro)
3. **NÃO PREEMPÇÃO** (Ninguém larga o Garfo)
4. **ESPERA CIRCULAR** (A rede de espera fecha o círculo)

SISTEMA OPERACIONAL (SO)



INTERVENÇÃO VIA ESCALONADOR PREEMPTIVO

```
while True:  
    a = 1
```

CPU (SEGURADA)

Disciplina: Sistemas Operacionais. Laboratório.

Alunos criam script loop. CPU monopolizada. O SO intervém via Escalonador Preemptivo, alocando fatias de tempo para processos como o mouse, quebrando a 'Exclusão Mútua Total' da CPU.

Script (Loop)

Script (Loop)

INTERRUPÇÃO (SO)

Outro Processo (ex: Mouse Driver)

Script (Loop)

EXCLUSÃO MÚTUA TOTAL DA CPU

MOUSE MOVE!
(Responsividade Garantida)

Evasão e o Algoritmo do Banqueiro

Se o **Deadlock** é o "crime", a **Evasão** é a inteligência que tenta impedir que ele aconteça analisando cada passo dos processos.

Definição Técnica

- **Evasão (Avoidance):** O Sistema Operacional **examina** constantemente os **pedidos de recursos**. Ele só concede um recurso se tiver a certeza de que o sistema continuará em um **Estado Seguro**.
- **Estado Seguro:** É uma situação onde existe pelo menos uma ordem de execução que **permite que todos os processos terminem sem causar um Deadlock**.

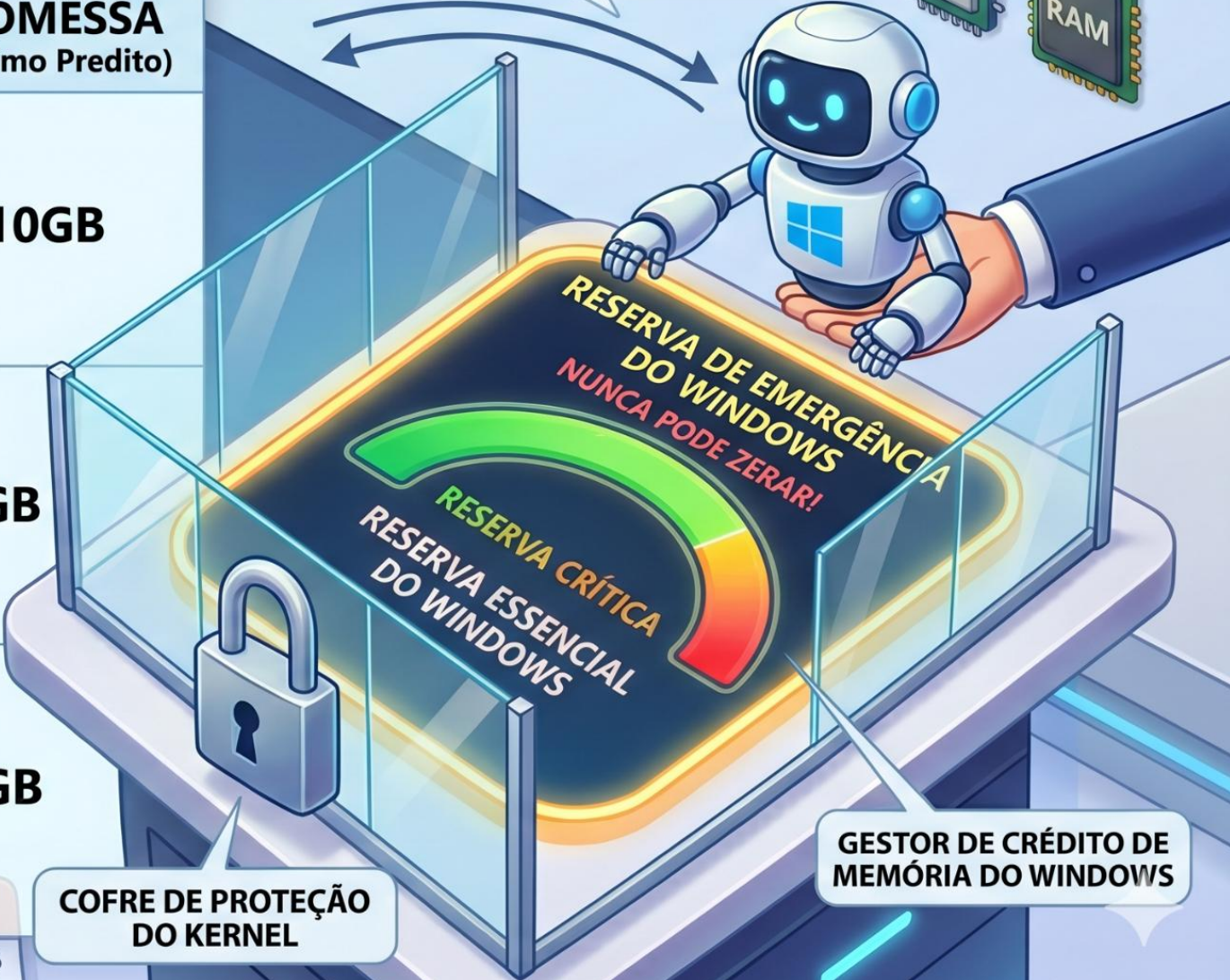
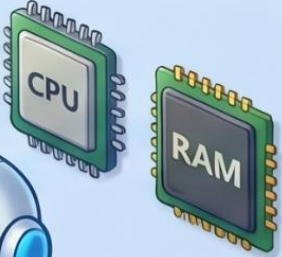
Definição Técnica

- **Algoritmo do Banqueiro:** Criado por Dijkstra, funciona como um gerente de banco que analisa se emprestar "crédito" (RAM/CPU) para um novo cliente não vai deixá-lo sem fundos para os clientes antigos que já estão operando.

Sugestão Visual: Tabela de 'Limite de Crédito de Memória'

A	B	C
PROGRAMA	USO ATUAL (Gasto)	PROMESSA (Máximo Predito)
 Chrome	 4GB	10GB
 Jogo (ex: Roblox/Fortnite)	 8GB	12GB
 Spotify	 1GB	2GB

Programas solicitam crédito.
Windows aprova o crédito de memória?



ESTOQUE: 10 FURADEIRAS TOTAIS

Bem-vindo à
Afogados
Ferramentas

10
FURADEIRAS
TOTAIS

AFOGADOS DA
INGAZEIRA
SEG, 20 ABR 2026
10:32 AM

PEDREIRO A:
MAX NECESSÁRIO:
8 FURADEIRAS |
FALTA SOLICITAR: 5

PEDREIRO B:
MAX NECESSÁRIO:
FURADEIRO: 5 FURADEIRAS
FALTA SOLICITAR: 3

NO/HOLD

A SOLICITA +5:
CONCEDER OU NEGAR?

O Banqueiro
da Obra

PEDIDO:
5 MAIS

O BANQUEIRO
(DONO DA LOJA)

PEDIDO:
5 MAIS

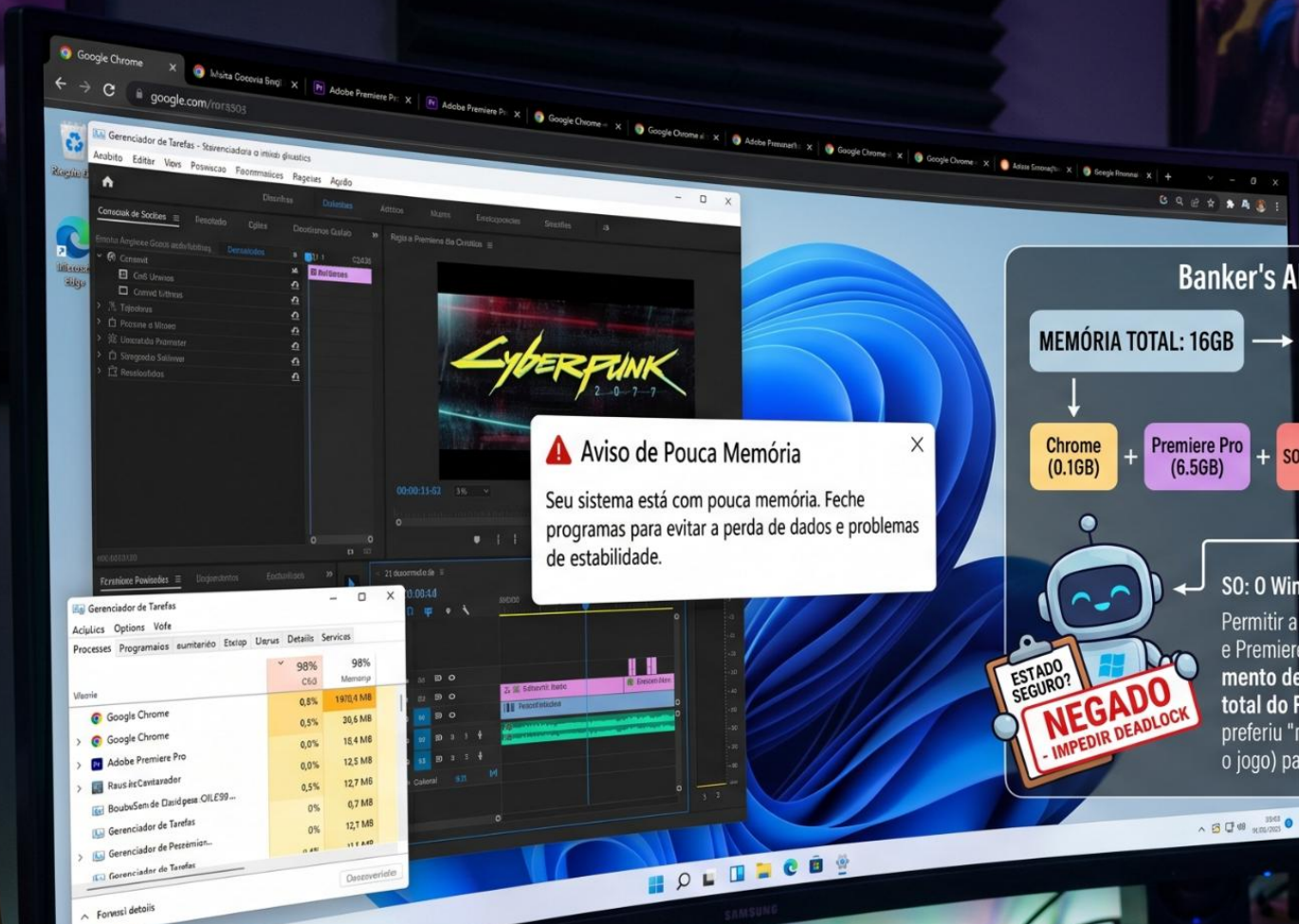
SE CONCEDER A (+5):

- ESTOQUE = 0
- B SOLICITA (+1)
- FALHA (BLOQUEIO)
- PEDREIRO B NÃO PODE TERMINAR! (DEADLOCK/IMPEDIMENTO) ?

SE NEGAR A (TEMPORARIAMENTE):

- ESTOQUE = 5
- PERMITIR B SOLICITAR
- B TERMINA E DEVOLVE (5) FURADEIRAS
- CONCEDER A PEDIDO
- A TERMINA
- TUDO SALVO!

USANDO:
3 UNIDADES



Aviso de Pouca Memória

Seu sistema está com pouca memória. Feche programas para evitar a perda de dados e problemas de estabilidade.

Banker's Algorithm



Detecção e Recuperação

Se a **prevenção falhou**, o Sistema Operacional precisa agir como um "detetive" e, se necessário, como um "carrasco" para **salvar a integridade do sistema**.

Definição Técnica

Detecção: É o processo em que o Sistema Operacional **monitora ativamente a execução dos programas** para identificar ciclos de espera. Ele procura por processos que **estão parados aguardando recursos que nunca serão liberados**.

Recuperação: É a ação corretiva para quebrar o impasse. O SO pode realizar a **Eliminação** (encerrar o programa travado) ou a **Preempção** (forçar a retirada de um recurso de um programa para entregá-lo a outro).

ESQUEMA VISUAL: DEADLOCK E QUEBRA DE CONEXÃO

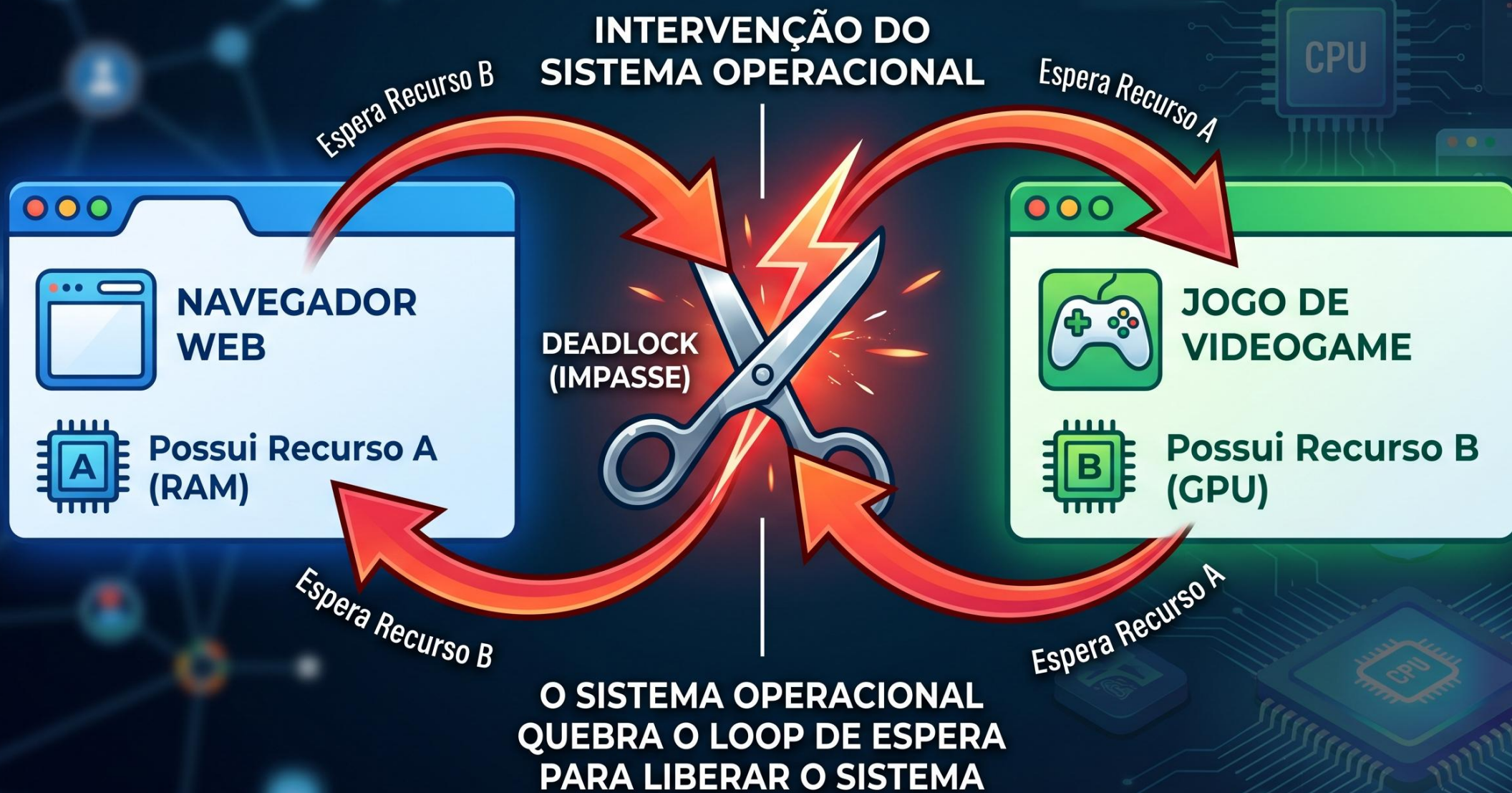


ILUSTRAÇÃO DA ANALOGIA: "O GUARDA NO CRUZAMENTO" (DEADLOCK EM SISTEMAS OPERACIONAIS)

SITUAÇÃO: O IMPASSE (DEADLOCK)



Quatro carros (Processos) chegam a um cruzamento (Recursos) ao mesmo tempo e se **bloqueiam** mutuamente em um círculo vicioso. Cada um precisa do espaço ocupado pelo outro.

DETECÇÃO: O OLHAR DO SISTEMA OPERACIONAL



RECUPERAÇÃO: SACRIFICANDO UM PROCESSO





Task Manager window showing system performance and running processes.

Nome	Status	CPU	Memória
Microsoft Word	42%	29%	
Biblioteca Exch	0.5%	5520.7 MB	637
Notificação Flan	0.5%	136.7 MB	
Processo Múltiplo de	0.1%	174.0 MB	
Condições de	0.2%	51.6 MB	
Condições de	5.5%	11.1 MB	
Condições de	19%	0.2	
Task Manager	0.1%	5	

Processos parciais (577)

Processo	CPU
Resposta de	0%
Condições de	0%
Resposta de	0%
Condições de	0%
Resposta de	0.7%
Condições de	6%
Resposta de	0%
Condições de	0%
Resposta de	0%

O Algoritmo do Avestruz

Às vezes, a melhor solução para um problema complexo é simplesmente ignorá-lo. Este é o conceito que justifica por que nossos computadores ainda travam de vez em quando.

Definição Técnica

Conceito: É a estratégia de **ignorar o problema**. O nome deriva da lenda (biologicamente falsa, mas computacionalmente útil) de que o avestruz enterra a cabeça na areia quando vê um perigo, **fingindo que ele não existe**.

Definição Técnica

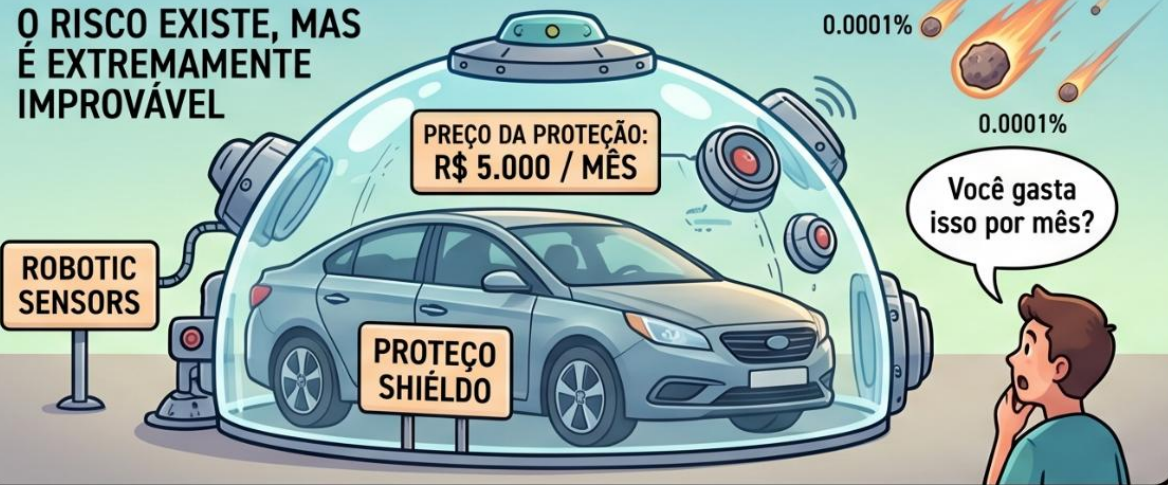
Justificativa Econômica: Sistemas Operacionais modernos (Windows, Linux, macOS) utilizam essa **estratégia para Deadlocks que ocorrem raramente**. O custo de tempo e processamento para evitar *todo e qualquer* impasse deixaria o computador lento demais para o usuário comum.

Segundo Tanenbaum: É mais razoável aceitar um **travamento raro** do que pagar o preço de um sistema extremamente pesado que tenta ser perfeito o tempo todo.

ILUSTRAÇÃO DA ANALOGIA: “O SEGURO CONTRA METEORITOS”

CENÁRIO RIDÍCULO: PROTEÇÃO EXCESSIVA

O RISCO EXISTE, MAS É EXTREMAMENTE IMPROVÁVEL



A “ESTRATÉGIA DO AVESTRUZ”: ECONOMIA

IGNORAR O RISCO E ECONOMIZAR



A RECUPERAÇÃO BARATA: REINÍCIO

ACEITAR O PREJUÍZO E “REINICIAR”

É MAIS BARATO DO QUE A PROTEÇÃO CONTÍNUA.



O EVENTO RARO: IMPACTO (DEADLOCK)

SE UM METEORITO CAIR (O DEADLOCK RARO)...

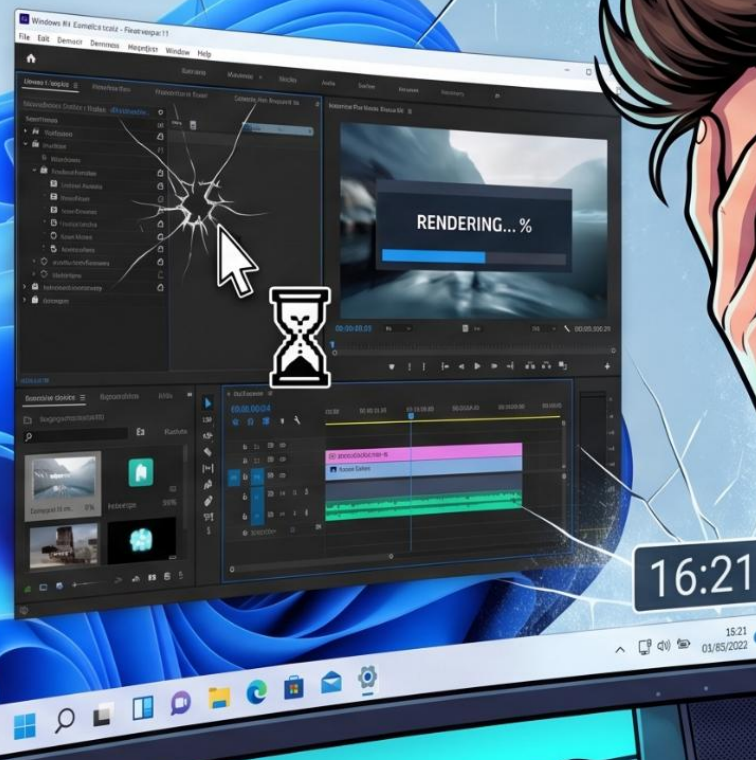


ALGORITMO DO AVESTRUZ:

O Windows ignorou o risco de impasse para manter o sistema rápido!



O sistema "não viu" o conflito e parou de responder.



A ÚNICA SAÍDA:
Intervenção Física:
BOTÃO RESET OU POWER.

O S.O. não pode se recuperar sozinho.



Nesse caso, o
"Avestruz" falhou.
É o preço da agilidade
nos outros 99%.

arineto1.github.io/ifpe2026